

PROJECT REPORT

Project Title: FlowOrchestrator: A CLI-Based Task Management System

Course Context: Core Java / Object-Oriented Programming (Flipped Course Project)

Student Name: Yerrapragada Sai Jaya Ram

Registration Number: 25BAI10286

Institution: Vellore Institute of Technology Bhopal University

Repository: <https://github.com/jayaramyerrapragada15-Triplet/FlowOrchestrator>

Language / Runtime: Java 17, Apache Maven

Submission Type: Individual Software Engineering Project

Abstract

The FlowOrchestrator is a headless, command-line Java application that enables a user to create, prioritize, track, and persist work items — “tasks” — across sessions without relying on a database server or graphical toolkit. The system is architected in four cleanly separated layers: a domain model layer (Task, Priority, TaskStatus), a custom exception layer distinguishing expected from exceptional failure modes, a persistence layer (FileStorage) built on standard buffered file I/O, and a service layer (TaskManager) that combines a HashMap and a PriorityQueue to provide both constant-time lookup by identifier and efficient priority-ordered retrieval. A thin presentation layer (Main) owns all Scanner-based interaction, input validation, and formatted output, keeping business logic free of I/O concerns. The result is a compact but production-quality demonstration of core Java competencies: encapsulation, custom checked/unchecked exceptions, the Collections Framework, the Streams API, and file-based state persistence.

Problem Statement s Objectives

Problem Statement

Individuals and small teams routinely need to track a backlog of work items that vary in urgency and move through a predictable lifecycle. Spreadsheets and sticky notes do not enforce structure, do not compute priority ordering automatically, and do not scale past a handful of items. A lightweight, dependency-free tool that runs anywhere a JVM is available — inside a CI container, over SSH, in a restricted terminal environment — fills this gap without the operational overhead of a full web application or database.

Objectives

- Design a task domain model rich enough to support priority, lifecycle status, and due dates, while remaining simple to serialize.
- Provide $O(1)$ average-case lookup of any task by identifier, and efficient retrieval of the single most urgent task at any time.
- Guarantee that invalid input (blank titles, malformed dates, unknown enum values) is caught and reported without crashing the session.
- Persist all state to local disk using only the standard library, so the tool requires no external services.
- Demonstrate, in a single cohesive codebase, the specific Core Java competencies required by the course: OOP architecture, custom exceptions, the Collections Framework, and the Streams/Lambda API.

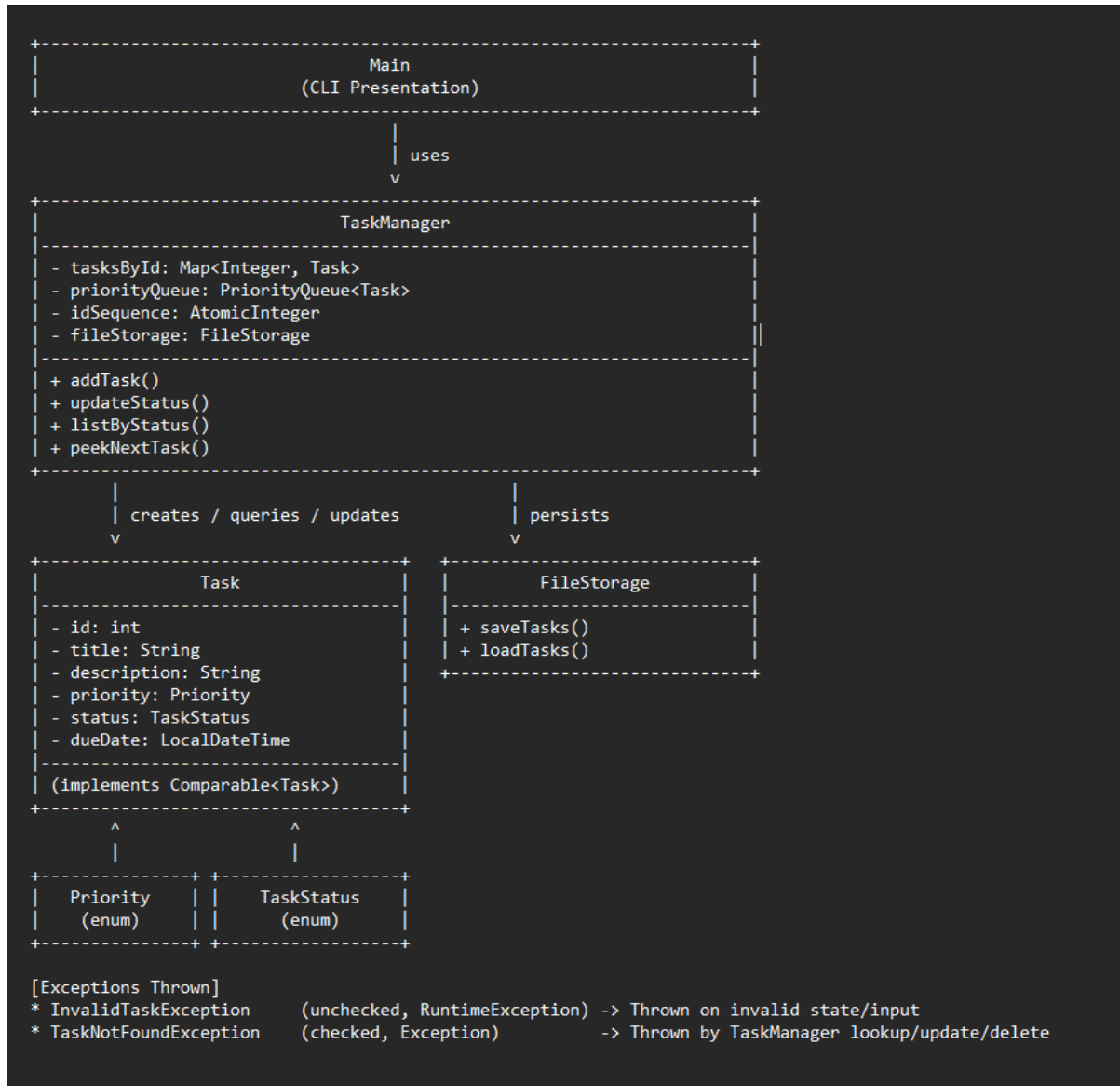
System Design & Architecture

Package Structure

Package	Responsibility
com.workflow	CLI entry point and interaction loop (Main)
com.workflow.model	Domain types: Task, Priority, TaskStatus
com.workflow.exception	TaskNotFoundException (checked), InvalidTaskException (unchecked)
com.workflow.util	FileStorage — buffered file persistence
com.workflow.service	TaskManager — business logic and in-memory indexing

- This is a classic layered architecture: presentation depends on service, service depends on model and persistence, and model/exception have no outward dependencies at all. Dependencies flow in one direction only, which keeps the domain model reusable and independently testable.

ASCII Class s Component Diagram



OOP Principles Applied

- Encapsulation: Task exposes only validated mutators (setTitle, setPriority, etc.); every field is private, and invariants (non-blank title, non-null priority/status) are enforced inside the setters and constructors themselves, not by callers.
- Abstraction: TaskManager exposes a small, intention-revealing API (addTask, updateStatus, listByStatus, peekNextTask, ...) that hides the dual HashMap

PriorityQueue indexing strategy entirely from the CLI layer. Main never touches a Map or PriorityQueue directly.

- Interface Segregation / Polymorphism: Task implements Comparable<Task>, allowing it to be dropped directly into a PriorityQueue<Task> and sorted via Comparator.naturalOrder() without any manual comparator wiring at call sites. Java enums (Priority, TaskStatus) act as small closed polymorphic hierarchies with per-constant behavior (getWeight(), getDisplayName()).
- Inheritance: The custom exception types extend Exception and RuntimeException respectively, inheriting the standard exception contract while adding domain-specific fields (TaskNotFoundException carries the offending taskId).

Design Rationale: HashMap + PriorityQueue

PriorityQueue in the JDK does not support efficient arbitrary-element removal or re-prioritization (remove(Object) is $O(n)$, and there is no decrease-key operation). Rather than implement a custom indexed heap — disproportionate complexity for a single-user CLI tool — TaskManager treats the queue as a derived index: the HashMap<Integer, Task> is the single source of truth, and the queue is rebuilt in $O(n \log n)$ whenever a task's priority changes or a task is deleted. Simple additions are $O(\log n)$ as usual. This trade-off is explicitly documented in the TaskManager Javadoc and is appropriate given the expected data scale (a personal or small-team backlog, not a distributed job scheduler).

Implementation Details

Data Persistence

FileStorage persists tasks to a flat file (tasks.dat) using BufferedWriter / BufferedReader over UTF-8, with one task per line in a pipe-delimited format:

Plaintext

```
id|title|description|priority|status|createdAt|dueDate
```

Because user-supplied titles and descriptions may themselves contain the pipe character or newlines, FileStorage escapes \, |, \n, and \r on write and reverses the escaping on read, so record boundaries can never be corrupted by free-text input. A missing storage file is treated as an empty task set (first run), while a structurally malformed line raises an IOException that identifies the offending line number, rather than silently dropping or misinterpreting data.

Custom Exceptions

Two custom exception types encode two distinct failure semantics:

- `TaskNotFoundException` (checked, extends `Exception`) — thrown when an operation references a task ID that does not exist. This is modeled as a checked exception because looking up a stale or mistyped ID is an expected outcome of normal CLI use, and callers are deliberately forced to handle it explicitly.
- `InvalidTaskException` (unchecked, extends `RuntimeException`) — thrown when constructing or mutating a `Task` would violate its invariants, or when parsing a `Priority` / `TaskStatus` from free-text input fails. This is modeled as unchecked because it represents an input-validation failure that every mutator can raise; requiring it on every method signature would add noise without improving safety, and the CLI layer already wraps each command in a try/catch that reports it cleanly.

Collections s Streams Usage

- `HashMap<Integer, Task>`: $O(1)$ average lookup, insertion, and removal by task ID inside `TaskManager`.
- `PriorityQueue<Task>`: Priority-ordered retrieval of the single most urgent task via `peekNextTask()`, backed by `Task.compareTo`.
- Streams API: `listByStatus`, `listByPriority`, `listOverdueTasks`, and `searchTasks` are all implemented as `stream().filter().sorted().collect()` pipelines; `getStatusSummary` uses `Collectors.groupingBy` combined with `Collectors.counting()` to build a status → count map in a single pass.
- Lambda Expressions: Method references (`Task::getId`, `Task::isOverdue`) and inline comparators (`Comparator.comparing(Task::getDueDate)`) are used throughout the query methods in place of anonymous inner classes.

CLI Layer

Main owns a single `run()` loop that prints a numbered menu, reads a selection, and dispatches to one of thirteen command flows via a switch expression. Every numeric, enum, and date input is parsed through a dedicated `promptX` helper (`promptInt`, `promptPriority`, `promptStatus`, `promptOptionalDateTime`) that loops until valid input is supplied, so malformed input never terminates the session. Domain exceptions raised by `TaskManager` are caught once at the top of the loop and rendered as friendly, labeled messages (`[Validation Error] ...`, `[Not Found] ...`, `[I/O Error] ...`).

Validation s Test Cases

The following scenarios were manually exercised against the compiled CLI to validate correct behavior. Each row lists the action taken, the input supplied, and the observed result.

#	Scenario	Input	Expected Result	Observed Result
1	Add task with valid data	Title, description, HIGH, valid due date	Task created with next sequential ID	Pass
2	Add task with blank title	Empty string at title prompt	Re-prompted; blank title rejected	Pass
3	Add task with invalid priority	URGENT (not a valid enum)	[Validation Error] / re-prompt listing valid options	Pass
4	Add task with malformed due date	15/09/2026	Re-prompted with expected format example	Pass
5	View task by valid ID	Existing ID	Full task details printed	Pass
6	View task by non-existent ID	999	[Not Found] No task exists with ID #999.	Pass
7	Update status through full lifecycle	PENDING → IN_PROGRESS → COMPLETED	Status transitions applied, reflected in listings	Pass

#	Scenario	Input	Expected Result	Observed Result
8	Update priority and re-check ordering	Change a LOW task to CRITICAL	Task moves to top of "list by priority" and becomes peekNextTask() result	Pass
G	Delete task with confirmation declined	n at confirm prompt	Task retained, "Deletion cancelled." shown	Pass
10	Delete task with confirmation accepted	y at confirm prompt	Task removed; subsequent lookup raises TaskNotFoundException	Pass
11	Search by keyword (case-insensitive)	Lowercase keyword matching a title with mixed case	Matching tasks returned	Pass
12	Filter by status with no matches	Status with zero tasks	(no tasks to display) shown, not an error	Pass
13	Overdue detection	Task with due date in the past, status PENDING	Task appears in overdue list with ** OVERDUE ** marker	Pass
14	Overdue exclusion for terminal states	Same as above but status COMPLETED	Task does not appear in overdue list	Pass

#	Scenario	Input	Expected Result	Observed Result
15	Persistent state across restarts	Add 2 tasks, save C exit, relaunch	"Loaded 2 task(s)" on next startup; data intact	Pass
16	Corrupt storage file handling	Manually truncate a line in tasks.dat	Startup fails fast with a line-numbered IOException message instead of silent data loss	Pass
17	Status summary counts	Mixed set of tasks across all four statuses	Per-status counts sum to total task count	Pass

Plagiarism s Integrity Declaration

I declare that the design, source code, documentation, and this report submitted for the FlowOrchestrator project represent my own original work, produced specifically for this course assignment. Standard Java language constructs, JDK library APIs (e.g., java.util, java.io, java.time), and Apache Maven build conventions are used in accordance with their public documentation; no third-party source code, templates, or AI-generated submissions from other students were copied or incorporated. Any external references consulted during development (official Oracle Java documentation, Apache Maven documentation) informed understanding of standard library behavior only and did not supply copied implementation code. I understand that this submission will be assessed for the integrity of authorship consistent with the institution's academic honesty policy.

Signed: Yerrapragada Sai Jaya Ram

Registration Number: 25BAI10286

Date: 18-09-2026